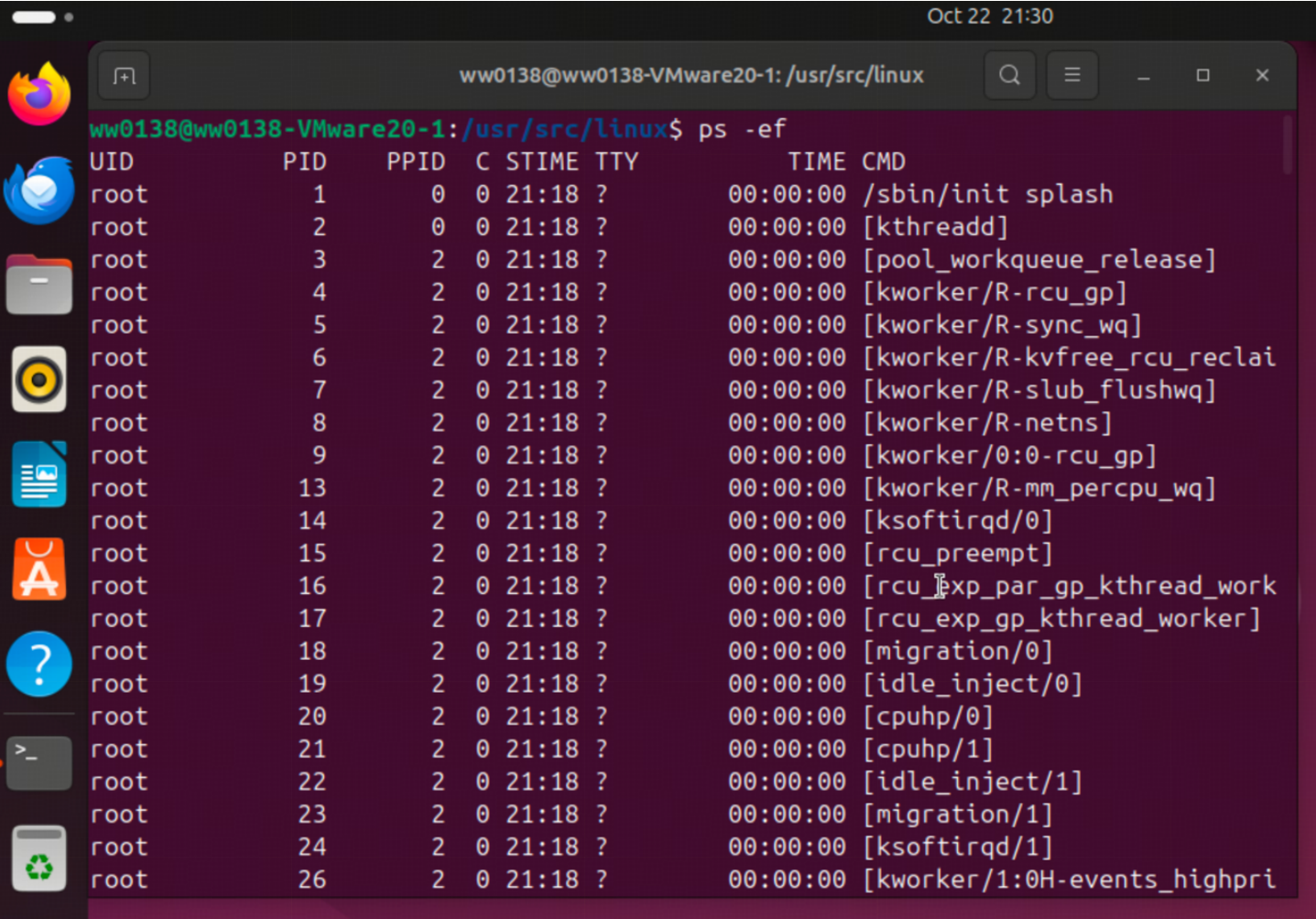


Lab04-PS-In-Usermode

目标

写一个C/C++程序，列出所有在用户模式下的进程：



工具

GCC编译器

代码块

```
1 sudo apt-get install build-essential
```

IDE

- 1. (推荐)Code::Block

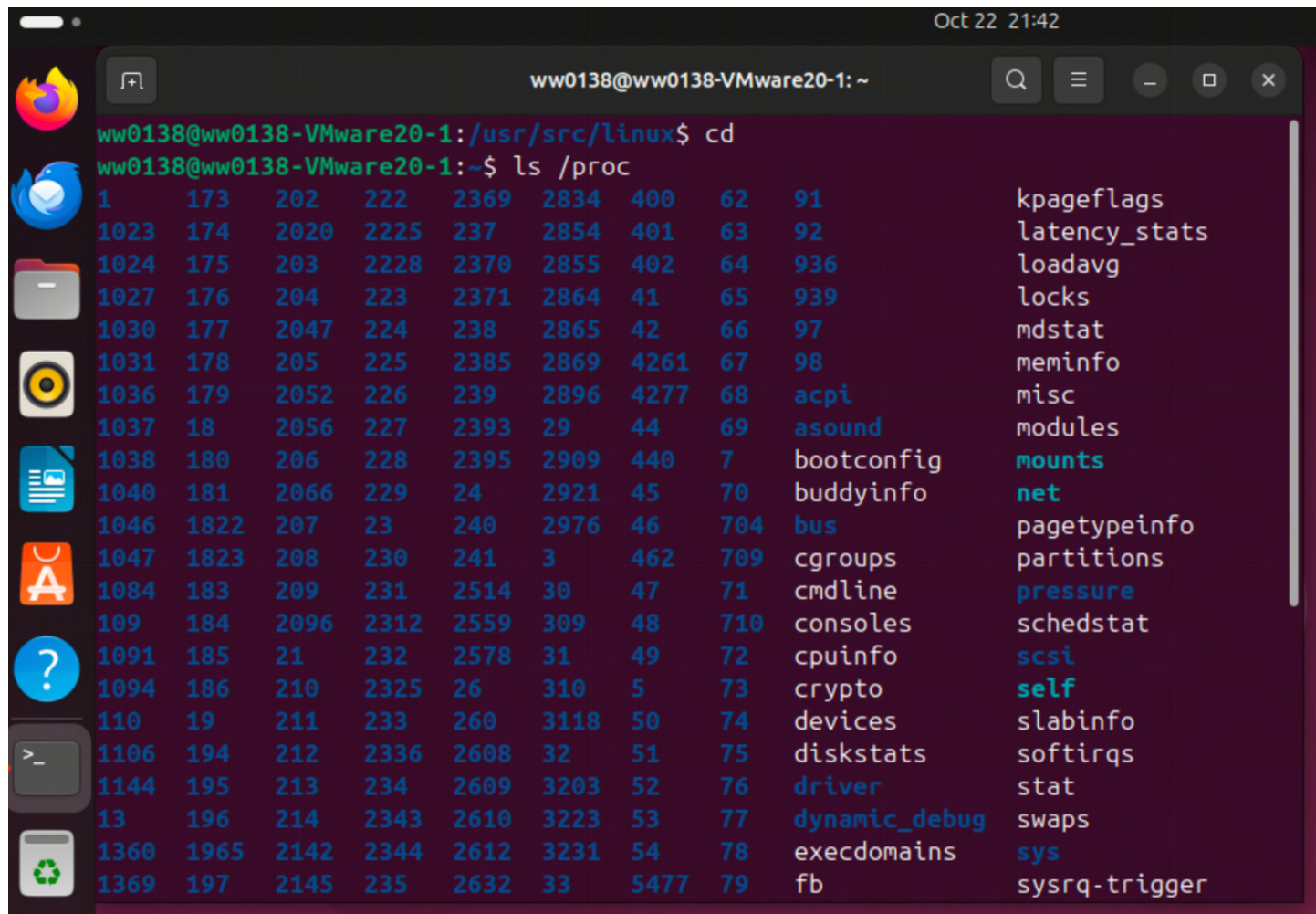
代码块

```
1 sudo apt-get install codeblocks
```

2. vim + gcc

所需信息

进程相关信息都放在/proc目录下



A terminal window titled 'ww0138@ww0138-VMware20-1: ~' showing the command 'ls /proc' and its output. The output is a long list of files and directories in /proc, including kpageflags, latency_stats, loadavg, locks, mdstat, meminfo, misc, modules, mounts, net, pagetypeinfo, partitions, pressure, schedstat, scsi, self, slabinfo, softirqs, stat, swaps, sys, and sysrq-trigger. The files are listed in a grid-like format with columns of numbers and names.

```
ww0138@ww0138-VMware20-1:/usr/src/linux$ cd
ww0138@ww0138-VMware20-1:~$ ls /proc
1      173    202    222    2369   2834   400    62    91      kpageflags
1023   174    2020   2225   237    2854   401    63    92      latency_stats
1024   175    203    2228   2370   2855   402    64    936     loadavg
1027   176    204    223    2371   2864   41     65    939     locks
1030   177    2047   224    238    2865   42     66    97      mdstat
1031   178    205    225    2385   2869   4261   67    98      meminfo
1036   179    2052   226    239    2896   4277   68    acpi    misc
1037   18     2056   227    2393   29     44     69    asound  modules
1038   180    206    228    2395   2909   440    7     bootconfig mounts
1040   181    2066   229    24     2921   45     70    buddyinfo net
1046   1822   207    23     240    2976   46     704   bus     pagetypeinfo
1047   1823   208    230    241    3     462    709   cgroups partitions
1084   183    209    231    2514   30     47     71    cmdline pressure
109     184    2096   2312   2559   309    48     710   consoles schedstat
1091   185    21     232    2578   31     49     72    cpuinfo scsi
1094   186    210    2325   26     310    5     73    crypto  self
110    19     211    233    260    3118   50     74    devices slabinfo
1106   194    212    2336   2608   32     51     75    diskstats softirqs
1144   195    213    234    2609   3203   52     76    driver  stat
13     196    214    2343   2610   3223   53     77    dynamic_debug swaps
1360   1965   2142   2344   2612   3231   54     78    execdomains sys
1369   197    2145   235    2632   33     5477   79    fb      sysrq-trigger
```

某个进程的具体信息：

```
Oct 22 21:45
ww0138@ww0138-VMware20-1: /proc/2362
ww0138@ww0138-VMware20-1:/$ cd /proc/2362
ww0138@ww0138-VMware20-1:/proc/2362$ ls
attr          fd            mem           personality   statm
autogroup     fdinfo        mountinfo     projid_map   status
auxv          gid_map       mounts        root          syscall
cgroup        io            mountstats    sched         task
clear_refs   ksm_merging_pages net            schedstat    timens_offsets
cmdline       ksm_stat      ns            sessionid     timers
comm          latency       numa_maps     setgroups    timerslack_ns
coredump_filter limits        oom_adj       smaps         uid_map
cwd           loginuid      oom_score     smaps_rollup  wchan
environ       map_files     oom_score_adj stack          stat
exe           maps          pagemap
ww0138@ww0138-VMware20-1:/proc/2362$
```

status文件:


```
ww0138@ww0138-VMware20-1: /proc/2362$ more status
Name:   gsd-media-keys
Umask:  0002
State:  S (sleeping)
Tgid:   2362
Ngid:   0
Pid:    2362
PPid:   1974
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 128
Groups: 4 24 27 30 46 100 114 1000
NSTgid: 2362
NSpid:  2362
NSpgid: 2362
NSsid:  2362
Kthreed:      0
VmPeak:      602464 kB
VmSize:      602320 kB
VmLck:        0 kB
VmPin:        0 kB
VmHWM:        25276 kB
VmRSS:        25276 kB
```

cmdline文件:

```
ww0138@ww0138-VMware20-1: /proc/2362$ cat cmdline
/usr/libexec/gsd-media-keysww0138@ww0138-VMware20-1: /proc/2362$
```

stat文件:

```
ww0138@ww0138-VMware20-1: /proc/2362$ cat stat
2362 (gsd-media-keys) S 1974 2362 2362 0 -1 4194304 2265 0 0 0 5 2 0 0 20 0 5 0
2274 616775680 6292 18446744073709551615 191989301706752 191989301884892 2814743
91438096 0 0 0 0 4096 16384 0 0 0 17 1 0 0 0 0 0 191989301961656 191989301969892
191989749989376 281474391439848 281474391439876 281474391439876 281474391441372
0
ww0138@ww0138-VMware20-1: /proc/2362$
```

字段列表（核心字段说明）

假设文件内容格式如下（以空格分隔）：`pid comm state ppid pgrp session tty_nr
tpgid flags minflt cminflt majflt cmajflt utime stime cutime cstime`

priority nice num_threads itrealvalue starttime vsize rss rsslim
startcode endcode startstack kstkesp kstkeip signal blocked sigignore
sigcatch wchan nswap cswap exit_signal processor rt_priority policy
delayacct_blkio_ticks guest_time cguest_time

序号	字段名	含义说明
1	pid	进程 ID（Process ID），唯一标识进程的数字。
2	comm	进程命令名，被括号包裹（如 (bash)），不包含命令行参数。
3	state	进程状态：- R：运行中（Runnable）- S：睡眠中（Interruptible Sleep）- D：不可中断睡眠（Uninterruptible Sleep，通常等待 IO）- Z：僵尸进程（Zombie）- T：暂停（Stopped）或跟踪中（Traced）
4	ppid	父进程 ID（Parent PID），创建该进程的进程 ID。
5	pgrp	进程组 ID（Process Group ID），进程所属的进程组编号。
6	session	会话 ID（Session ID），进程所属的会话编号（用于终端管理）。
7	tty_nr	终端设备号（TTY device number），进程关联的终端设备编号（0 表示无终端）。
8	tpgid	终端进程组 ID（TTY Process Group ID），与进程终端关联的进程组 ID（-1 表示无）。
9	flags	进程标志（Flags），位图形式的内核标志（如是否为内核线程、是否被跟踪等）。
10	minflt	次要页错误次数（Minor Page Faults），不需要从磁盘加载数据的页错误（如内存映射未命中）。
11	cminflt	所有子进程的次要页错误总和（Cumulative minflt）。
12	majflt	主要页错误次数（Major Page Faults），需要从磁盘加载数据的页错误。
13	cmajflt	所有子进程的主要页错误总和（Cumulative majflt）。
14	utime	用户态 CPU 时间（User Time），进程在用户态运行的时间（单位：系统时钟滴答）。
15	stime	内核态 CPU 时间（System Time），进程在内核态运行的时间（单位：系统时钟滴答）。
16	cutime	所有子进程的用户态 CPU 时间总和（Cumulative utime）。
17	cstime	所有子进程的内核态 CPU 时间总和（Cumulative stime）。
18	priority	动态优先级（Dynamic Priority），内核调度时使用的优先级（值越小优先级越高）。
19	nice	静态优先级（Nice Value），用户可调整的优先级（范围 - 20~19，值越小优先级越高）。
20	num_threads	进程包含的线程数（Number of Threads）。

21	itrealvalue	间隔定时器剩余时间（已废弃，通常为 0）。
22	starttime	进程启动时间（Start Time），从系统启动到进程启动的时间（单位：系统时钟滴答）。
23	vsize	虚拟内存大小（Virtual Size），进程使用的虚拟内存总量（单位：字节）。
24	rss	常驻内存大小（Resident Set Size），进程实际占用的物理内存页数（单位：页，1 页 = 4KB 通常）。
25	rsslim	进程可使用的最大 RSS 限制（单位：字节）。
26	startcode	程序代码段的起始地址（如 ELF 文件的.text 段）。
27	endcode	程序代码段的结束地址。
28	startstack	进程栈的起始地址。
29	kstkesp	内核栈指针（Kernel Stack Pointer），进程在内核态时的栈顶地址。
30	kstkeip	内核指令指针（Kernel Instruction Pointer），进程在 kernel 态时的下一条指令地址。
31+	其他字段	包括信号相关（signal/blocked等）、调度策略（policy）、IO 等待时间等，通常用于内核级调试。

实验流程

1. 写一个c程序列出所有用户模式的进程，结果跟ps的命令类似（包含UID, PID, PPID, C, STIME, TTY, TIME, CMD）：

```
Oct 22 21:30
ww0138@ww0138-VMware20-1: /usr/src/linux
ww0138@ww0138-VMware20-1: /usr/src/linux$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root          1        0  0  21:18 ?           00:00:00 /sbin/init splash
root          2        0  0  21:18 ?           00:00:00 [kthreadd]
root          3        2  0  21:18 ?           00:00:00 [pool_workqueue_release]
root          4        2  0  21:18 ?           00:00:00 [kworker/R-rcu_gp]
root          5        2  0  21:18 ?           00:00:00 [kworker/R-sync_wq]
root          6        2  0  21:18 ?           00:00:00 [kworker/R-kvfree_rcu_reclai
root          7        2  0  21:18 ?           00:00:00 [kworker/R-slub_flushwq]
root          8        2  0  21:18 ?           00:00:00 [kworker/R-netns]
root          9        2  0  21:18 ?           00:00:00 [kworker/0:0-rcu_gp]
root         13        2  0  21:18 ?           00:00:00 [kworker/R-mm_percpu_wq]
root         14        2  0  21:18 ?           00:00:00 [ksoftirqd/0]
root         15        2  0  21:18 ?           00:00:00 [rcu_preempt]
root         16        2  0  21:18 ?           00:00:00 [rcu_exp_par_gp_kthread_work
root         17        2  0  21:18 ?           00:00:00 [rcu_exp_gp_kthread_worker]
root         18        2  0  21:18 ?           00:00:00 [migration/0]
root         19        2  0  21:18 ?           00:00:00 [idle_inject/0]
root         20        2  0  21:18 ?           00:00:00 [cpuhp/0]
root         21        2  0  21:18 ?           00:00:00 [cpuhp/1]
root         22        2  0  21:18 ?           00:00:00 [idle_inject/1]
root         23        2  0  21:18 ?           00:00:00 [migration/1]
root         24        2  0  21:18 ?           00:00:00 [ksoftirqd/1]
root         26        2  0  21:18 ?           00:00:00 [kworker/1:0H-events_highpri
```

- 程序通过读取 `/proc` 虚拟文件系统来获取进程信息，主要读取的文件包括：
 - `/proc/[pid]/stat`：包含进程的基本状态信息
 - `/proc/[pid]/cmdline`：包含进程的命令行参数
 - `/proc/uptime`：用于计算进程启动时间

2. 一个遍历目录的例子

a. 撰写程序listdir.c

代码块

```
1  #include <stdio.h>
2  #include <dirent.h>
3  #include <unistd.h>
4  #include <stdlib.h>
5
6  int main()
7  {
8      DIR * dir;
9      struct dirent * ptr;
10     /*open dir*/
11     dir = opendir("/home");
12     /*read dir entry*/
13     while((ptr = readdir(dir)) != NULL)
```



```
14     {
15         printf("d_name : %s", ptr->d_name);
16         if (ptr->d_type==DT_DIR){
17             printf("\tDir");
18         }
19         printf("\n");
20     }
21     /*close dir*/
22     closedir(dir);
23     exit(0);
24 }
```

b. 编译程序

代码块

```
1 gcc listdir.c -o listdir
```

c. 运行

代码块

```
1 ./listdir
```

3. 撰写实验报告。（考虑到难度，大家优先实现python版本即可，感兴趣的可自行尝试C语言版本）